

计组 · 薄弱点背诵手册

408 计算机组成原理 · 错题血点地图

用途：每天早读 10 分钟过一遍，每条都配记忆钩子，背熟一条在打卡表记一笔。

来源：王道小程序错题重练卷（记录 3400 / 3401 / 3402 + 补录，共 93 题）暴露的薄弱模型。

创建：2026-09-01 · 随对话复习暴露的新薄弱点持续追加。

背诵法则：不背"这道题选什么"，只背 "这类题的判断式 + 记忆钩"。

术语	含义	记忆钩
按字节编址	每个字节(8位)分一个地址号；32位字长 = 连续4字节占1个字	门牌按房间发，一个字是4间连号套房
PC 程序计数器	存"下一条要取的指令的地址"	导航仪：指路用
IR 指令寄存器	存"刚取回来的那条指令"（位数 = 指令字长）	菜谱当前页：正读的那行
MAR 地址寄存器	要访问的主存地址	Memory Address = 去哪家（取件码）
MDR 数据寄存器	从主存取回/写入的数据	Memory Data = 拿的货（包裹）
可见 / 透明	可见 = 能在指令里按名字读写；透明 = 硬件内部摸不到	可见是菜单上的菜，透明是后厨设备
汇编语言程序员	写汇编指令的人（比C程序员低一层，能碰寄存器、不能碰微指令）	能进前厅，进不了后厨
十六个1万能钥匙	16位全1：无符号 = 65535，有符号 = -1	同一串位模式，两张报关单
ax / imm	ax：16位通用寄存器（累加器）；imm：立即数，直接写在指令里的常数	imm = immediate，"明码标价"的常数
Intel指令格式	目的操作数在前：add ax, bx → ax = ax+bx；结果存进前者	"先写收货人"

1. 三种 I/O 方式的串行/并行特点

方式	CPU与外设	传送与主程序
程序查询	串行	串行
中断	并行	串行 （传送时仍要中断主程序）
DMA	并行	并行

📌 **记忆钩**：只有 DMA 是"双并行"；中断方式传送时主程序仍要让路，所以传送与主程序串行。

2. DMA 完整机制（四阶段）

预处理（CPU设控制器：起始地址/字数/方向，只设一次）→ 传送（DMA自搞外设↔内存，每字借一个存储周期）→ 后处理（DMA发中断，CPU收尾）

📌 **记忆钩**：CPU只管下单和验收，搬运全程外包。周期挪用三兄弟：停止访存（整段霸占）/ 周期挪用（每字借一个周期）/ 透明传送（CPU不用才借）。挪用借的是存储周期 = 总线周期。

3. 中断总开关与外设接口

- **允许中断触发器**：1位总开关，=1（开中断）才理会外中断请求，=0（关中断）忽略（内中断/非屏蔽不受影响）。响应时硬件自动关，服务程序自己开。钩子：**开=竖天线，关=免打扰**
- **中断隐指令**：不在指令集里、硬件在响应周期自动完成：关中断→存断点→形成向量地址。钩子：**隐指令=硬件自动驾驶**

- **外设接口**：外设一侧的适配盒子，三件：数据缓冲寄存器（中转）、控制寄存器（收命令）、状态寄存器（报就绪/忙）
- **查中断**：每条指令结束时CPU挨个检查中断源有无请求；软件做叫中断查询，硬件做叫判优电路

4. 响应时机（必考对比）

- **中断等指令**：只能在一条指令执行完才响应
- **DMA等总线**：一个总线周期结束就响应；总线周期 $\approx$ 存储周期 $\approx$ 流水段长度

🔗 **记忆钩**：中断要的是 CPU 的"人"，等它干完手头的指令；DMA 要的只是"总线"这条路，一个周期结束就让路。流水线问"第k级来了DMA请求"→ 答"该级执行完后响应"。

5. 中断请求的目的（极易混）

方式	中断请求的目的
程序中中断	为了传送数据 （喊人来搬货）
DMA	为了报告传送结束 （打电话报平安，请CPU收尾）

6. 优先级（三种别混）

名称	谁决定	管什么
请求优先级	硬件排队线路（固定）	物理排队
响应优先级	判优电路/查询程序	谁先开始执行
处理优先级	中断屏蔽字（屏蔽标志）	谁能打断谁 = 多个服务程序执行完的次序

🔗 **记忆钩**：屏蔽字管"插队"（处理），判优电路管"排队"（响应）。外加：DMA优先级 > 外中断（防高速设备丢数据）；关中断时内中断和非屏蔽中断仍可响应。屏蔽字只动第三种。

7. 中断响应周期三件事

关 存 发：关中断 → 保存断点 → 发响应信号并形成向量地址（全部由中断隐指令完成）

- 为什么先关中断：不关就可能丢断点（类比 PV 操作的原子性）
- 中断向量地址 = 中断服务程序入口地址的地址（地址的地址！）
- 恢复现场前的关中断由 关中断指令 完成，别和隐指令混

🔗 记忆钩：先锁门（关中断）再记门牌（存断点）最后发信号指路（发向量地址）。

8. I/O 杂项速记

- I/O 接口三寄存器：数据缓冲、控制、状态（没有指令寄存器、没有 MAR）
- I/O 设备经 设备控制器 连系统总线（不是 DMA 控制器）
- 磁盘地址 = 磁道号（柱面号）+ 磁头号（盘面号）+ 扇区号；写盘是 串行
- 统一编址用 访存指令 做 I/O；独立编址才用专门 I/O 指令
- 周期挪用：每传一个数据占用一个 存储周期

0. CPU 全家福与基础定义

部门	成员
指挥部（控制器）	IR、PC、指令译码器、操作控制器
工厂（运算器）	ALU、状态寄存器、通用寄存器、暂存寄存器
前台（与主存交接）	MAR、MDR
道路	内部总线 + 三态门
不属于CPU	地址译码器（主存的部件）——译码器住在内存楼

- **两个正交维度**：单周期/多周期 = 一条指令用几个时钟；单总线/多总线 = 内部几条数据通路。单总线撑不起单周期（单周期要并行，单总线一次一路）
- **组合逻辑(硬布线)控制器**：纯门电路现算控制信号，快但难改；微程序控制器查控制存储器，慢但灵活。控制信号来自：译码信号 + 时钟节拍（+标志反馈）
- **三态门**：0/1 之外还有高阻态(断开)，单总线同一时刻只许一个部件发言，其余切高阻。钩子：**带"闭嘴"档的开关**
- **间址周期**：间接寻址时插在取指与执行之间，专取"操作数地址"，结束后真地址在MDR。钩子：**间址=多跑一趟取真地址**
- **可见性阶梯**：通用寄存器 > PC（转移/调用间接改）> 状态寄存器（半透明）> IR/MAR/MDR（完全透明）。钩子：**菜单菜→导航仪→半开后厨→全后厨**

为什么有延迟：ALU等组合电路无记忆，信号逐级穿门需要时间；两个寄存器间最慢路径决定时钟周期下限：

$$\begin{aligned} T_{\text{时钟}} &\geq \text{最长组合逻辑延迟} & f_{\text{主频}} &= 1 / T_{\text{时钟}} & \text{执行时间} &= \text{指令数} \times \text{CPI} \times T_{\text{时钟}} \\ & & \text{MIPS} &= f / (\text{CPI} \times 10^6) \end{aligned}$$

1. PC 五连

问法	结论
转移指令中 PC 改几次	2 次 ：取指后默认+1（加一条指令长度），执行时转移目标 覆盖
PC 位数取决于	存储容量 + 指令字长 ，与机器字长无关
指令从主存读出依据	总是 根据 PC（转移也只是先把目标写进 PC，取指仍看 PC）；条件转移不成功时 PC 保持+1值不跳
汇编程序员可见的寄存器	PC （可经转移/调用设置）
全透明的寄存器	IR （硬件自动加载）；状态寄存器半透明

🔗 **记忆钩**：+1是例行公事，跳转是特殊覆盖；跳转只改导航目的地，开车还是看导航。位数口诀"容量定上限，指令字长定步长"（4GB/32位指令 → PC最少30位，取指左移2位还原）。顺序执行的"+1"是加一条指令的长度，不是恒定的1。

2. 其他寄存器速记

- 间址周期结束后，MDR 里是 **操作数地址**（不是操作数，更不是指令）
- CPU 不含 **地址译码器**（那是主存的部件）
- 控制部件 = **指令寄存器 + 程序计数器 + 操作控制器**；状态条件寄存器属于 **运算器**（存 CF/OF 等）
- 组合逻辑控制器：微操作信号 ← **指令译码信号 + 时钟**

3. 单周期 vs 多周期

- 单周期时钟由 **最慢指令** 决定；加复杂指令会拖垮主频 → 多周期才更易支持乘除法
- 单周期必须 **分离** 指令/数据存储器；多周期可共用 **单端口** 存储器（分时复用）
- 单周期专用通路多 → 冗余大、成本高；多周期部件复用 → 利用率高
- 不一定等于一个时钟周期的操作：**主存读**（内存慢）、单周期里的 **ALU运算**（要求"小于"而非"等于"）

4. 单总线结构

- 部件挂总线靠 **三态门**（控制通断的开关）
- ALU 只能 **一个输入端** 直连总线，另一输入经寄存器；输出经 **暂存器**（防结果经总线倒灌回输入）。钩子：**一进一存一出缓**

5. 流水线

- 数据冒险三解：**插 nop（暂停）/ 转发旁路 / 编译器调序**；旁路解决不了 **load-use**
- 插 nop 的原理：数据冒险 = 后一条要用前一条产生的数据；nop 延迟的是用数据的后一条，等前一条写回。钩子：**nop 给等米下锅的指令争取时间**
- 分支预测治的是 **控制冒险**，别选进数据冒险的题

冒险	一句话识别	药方
数据冒险	等数据：后面要用前面还没产生的结果	nop / 旁路 / 调序
结构冒险	抢设备：两条指令同时要用同一个部件	加硬件资源
控制冒险	迷路：分支没算完，不知道去哪取下一条	分支预测

6. 微程序控制器

微命令（一个控制信号）→ 微操作（执行的动作）→ 微指令（微命令的集合）→ 微程序（有序排列）→ 一条微程序实现一条机器指令

- 微程序类比：机器指令 = 菜单上的菜，微程序 = 菜谱，微指令 = 菜谱一行步骤，微命令 = 一个具体动作。方向口诀：**指令→程序→指令→命令**（从大到小）
- 控制存储器：**ROM** 实现，属于 **CPU**（不是主存！）
- **入口地址**：控存里存着所有指令的微程序，操作码指出本指令那段微程序的起始地址，微指令计数器从此开始执行。钩子：**操作码报菜名，入口地址=菜谱起始页码**
- 微指令计数器决定 **微指令** 执行顺序（不是机器指令顺序）
- 硬布线：快但难设计；微程序：慢但灵活

7. 异常四分法

类型	特点	例子
中断	外部异步事件	键盘输入
自陷	自愿的、故意的	系统调用
故障	可救活，处理后重执行该指令	缺页、访存校验错
终止	无法挽救，只能停	硬件毁灭性错误

🔗 记忆钩：故障能救活重来，终止直接拔管。状态寄存器归运算器（标志是工厂的产品）。

1. Cache 地址拆分三步法

1. **块内地址位数** = $\log_2(\text{块大小})$
2. **直接映射**：行号 = $(\text{主存地址} \div \text{块大小}) \% \text{行数}$ ；**组相联**：组号 = $(\text{主存地址} \div \text{块大小}) \% \text{组数}$ （组数 = 行数 $\div$ 路数）
3. 剩下的高位全是 **标记**

🔑 **记忆钩**：低给块内、中给行/组、高给标记——"低位分房，中位分区，高位发证"。

映射方式	地址结构
直接映射	标记 行号 块内地址
组相联	标记 组号 块内地址
全相联	标记 块内地址（无中间层）

2. 标记阵列大小

每行标记项 = 标记位 + 有效位（+ 脏位 + 替换算法位） 标记位 = 总地址位数 - 组号位 - 块内位

⚠ **陷阱**：全写法 + 随机替换 → 不需要脏位和替换算法控制位！例：主存是 Cache 的 $4096(=2^{12})$ 倍 → 标记12位；64块直接映射 → $64 \times (12+1) = 64 \times 13\text{bit}$ 。

脏位：标记 Cache 块是否被修改过，决定换出时要不要写回主存——只有 **回写法** 需要。

3. 写策略配对（死记）

时机	策略
命中	全写法 / 回写法
失效	写分配法 / 非写分配法

🔑 **记忆钩**：配对：写分配+回写；非写分配+全写。失效时"把块拉进来改"（写分配）才有"弄脏了要回写"；"不拉进来"（非写分配）就直接写主存（全写）。口诀：命中管全写回写，失效管分配。

4. 交互单位与 Cache 性质

- CPU ↔ Cache（或主存）：**字**；Cache ↔ 主存：**块**（零售按字，批发按块）
- Cache 替换单位是 **块**；Cache 功能 **全由硬件** 实现；Cache 与主存地址空间 **相互独立**（不是统一编址）

5. 主存与扩展

- DRAM 地址线行列复用：引脚 +1 根 → 行数、列数各翻倍 → 容量 **× 4**
- 扩展判断：容量够位宽不够 → 位扩展；位宽够容量不够 → 字扩展；都不够 → 字位扩展（80386 以4B编址，8K × 8 芯片组 32KB = 8K × 32bit → **位扩展**）
- EPROM 可改写但 **不是** RAM（写入慢、次数有限）
- 多级存储只对 **应用级程序员** 透明；CPU 可与主存 **直接** 交换信息

6. 虚拟存储与三级命中

- 虚拟存储器：对 **应用程序员透明**，对系统程序员不透明；页表只能由 **OS 和硬件** 访问
- 命中组合铁律：**Cache 命中 → Page 必命中**；**TLB 命中 → Page 必命中** → "TLB 命中 + Cache命中 + Page未命中" **不可能发生**
- 三级存储（Cache/主存/硬盘）中，CPU 直接根据 **主存物理地址** 访问
- TLB 查表：页号 = 虚地址高位（位数 = $32 - \log_2 \text{页大小}$ ），页框号 + 页内偏移拼物理地址；**TLB命中只保证页在主存，不保证数据在Cache**

7. 概念补口（字/块、交叉编址、EPROM 家族）

概念	结论	记忆钩
地址位数	$= \log_2(\text{单元个数})$; $K=2^{10}$, $16K=2^{14} \rightarrow 14$ 位; " $\times 1/\times 8$ "是数据位宽, 与地址线无关	门牌数看房间数, 不看房间面积
字 vs 块	字=CPU存取的处理单位; 块=Cache $\leftrightarrow$ 主存的搬运单位 (含多个字)	零售按字, 批发按块
交叉编址	片选在地址 最低几位 ; 顺序编址片选在高位	交叉=门牌轮流发 (低位选片)
只读家族	PROM只写一次; EPROM紫外擦除可多次; EEPROM电擦除; 闪存基于E ² PROM	E=可擦除, 多一个E用电擦

1. C 语言类型转换黄金法则

转换不改变位模式，只改变解释（位模式是货物，类型是报关单——换单不换货）

- 等长转换（ $\text{short} \leftrightarrow \text{unsigned short}$ ）：位模式原样搬，负数变"大正数"。例：
 -32767 （补码 $1000\cdots 0001$ ） $\rightarrow \text{unsigned} = 2^{16} - 2^{15} + 1 = 32769$
- 变长转换（ $\text{short} \rightarrow \text{unsigned int}$ ）：先 **符号扩展** 到 32 位，再按无符号解释。
例： $-32767 \rightarrow \text{ui} = 2^{32} - 2^{15} + 1$
- $65535 \rightarrow \text{short} = -1$ （全 1 位模式按补码读）

2. 补码 $\leftrightarrow$ 移码

- 同一数值：移码 = 补码 **最高位取反**（数值位全同）。钩子：**翻头码 / 数轴搬家**（整体右移偏置，全变非负好比较）
- 相同位数 $\rightarrow$ 表示范围相同；**[0]补 $\neq$ [0]移**（ $00\cdots 0$ vs $10\cdots 0$ ），但各自表示唯一
- 移码表示浮点阶码，补码表示定点整数
- 寄存器全 1 = +127 $\rightarrow$ 只能是 **移码**（原/反/补的 +127 都是 01111111 ）
- 定义： $[x]\text{移} = 2^{n-1} + x$ ；算法两条路：真值+偏置，或补码翻最高位

3. 模4补码

- 存1算2看溢出**——存储只需 1 个符号位（两位总相同）；送 ALU 加减才扩成双符号位；双符号不同=溢出（01正溢/10负溢）
- 更易检查 **加减** 运算溢出（不是乘除）

4. 小端与进制

- 小端：**低字节放低地址**（机器码从左到右由低到高地址读时，立即数要反过来写：
 $-3 = \text{FFFDH} \rightarrow$ 写成 FD FF ）。钩子：**小端倒字节**

- 十进制转八进制：整数 **除基取余**（倒排），小数 **乘基取整**（顺排）； $137.5 \rightarrow 211.4$ 。钩子：除基取余倒着读，乘基取整顺着读；八进制答案里出现 8、9 直接排除

5. 运算标志位（四连问）

标志	看什么	钩子
OF	有符号运算溢出（最高位进位 $\oplus$ 次高位进位）	问"溢出吗"（只管有符号）
SF	结果最高位（符号）	问"正负吗"
CF	最高位有进位/借位	问"进位吗"（只管无符号）
ZF	结果是否为0	问"归零吗"

⚠ **陷阱：**有符号加法也会动 CF、无符号加法也会动 OF，只是"没意义"。两操作数符号不同必不溢出。

6. 补码读法与移码极值

- 负数补码还原：**取反加一现原形**（FFFDH $\rightarrow$ 取反0002， $+1=3 \rightarrow -3$ ）。负数位模式越接近全1越大（FFFF=-1 是负数老大）
- [0]移 永远是 **10...0**；全零 00...0 表示 -2^{n-1} （最小负数）
- 求极值口诀：**最大把1往高位堆，最小把1往低位堆；补码先留符号位，移码直接堆完减偏置**

1. 性能公式

$$\begin{aligned}\text{CPU执行时间} &= \text{指令条数} \times \text{CPI} \times \text{时钟周期} & \text{MIPS} &= \text{主频} \div (\text{CPI} \times 10^6) \\ \text{主频} &= 1 \div \text{时钟周期}\end{aligned}$$

$$\text{平均CPI} = \Sigma(\text{各类指令条数} \times \text{该类CPI}) \div \text{指令总条数}$$

- **CPI与时钟频率无关**（频率只决定快慢，不决定每指令花几个周期）
- 两机 CPI 相等 $\neq$ 速度相等（指令条数、主频还可能不同）
- 科学计算看 **MFLOPS**（每秒百万次浮点运算，科学算浮点看它）

🔗 **记忆钩：**CPI 是"每条指令的步数"，主频是"步子迈多快"——步数和步速是两回事。

2. 编译流水线（顺序死记）

预处理 → 编译 → 汇编 → 链接 （备菜 → 翻译成方言 → 转机器话 → 零件拼装）

- 编译程序：一次全翻译，生成目标文件，编译慢运行快。钩子：**编译器拍合影**
- 解释程序：翻译一句执行一句，不生成目标文件，速度慢。钩子：**解释器搞直播**
- 汇编程序也是翻译程序（汇编语言 → 机器语言）

概念	结论	记忆钩
并行 vs 串行	高速总线用串行（多线并行信号到不齐、干扰大，频率上不去）	多车并排跑不齐，单车道高铁飙速度
突发传输	只报首地址，连续传 n 个字（地址自动递增）	报个门牌号，整条街快递全收
总线时钟	处理器总线/内存总线/PCIe/USB 各不同，时钟协议各管各的	不同等级公路，各有各的限速
字符帧	1 起始 + 数据位 + 校验位(可选) + 1 停止； 比特率 = 帧位数 $\times$ 每秒字符数	帧=信封，起始位开头戳， 停止位封口
读写命令	事务地址阶段随地址发出，告知从设备"取还是送"	地址=去哪间房，命令=送还是取
隐含寻址	地址不写明（默认累加器），省地址字段、缩短指令	话不说全，省格子
ISA	软硬件的合同：定义程序员可见的一切（指令集/格式/寄存器/寻址/大小端），不管硬件实现	x86和ARM是两种合同； Intel/AMD同签x86但工艺不同→软件兼容
完整计算机系统	= 硬件系统 + 软件系统（裸机+软件才完整）	裸机+软件=完整系统
层次顺序	高级语言→汇编→操作系统→机器语言（再下是微程序）	高汇操机：写高语→翻汇编→操管加载→机器执行
MFLOPS	每秒百万次浮点运算，科学计算的标尺	科学算浮点，看MFLOPS

1. 系统总线三分类方向（记反+漏地址总线的坑）

总线	方向	钩子
数据总线 DB	双向（读入/写出）	货能进能出
地址总线 AB	单向（CPU 只发不收）	只喊门牌不被喊
控制总线 CB	双向（命令出、状态应答回）	有问有答

🔗 记忆钩：地址总线位数 $n \rightarrow$ 寻址 2^n 个单元。地址单行、数据双行、控制有来有回。

2. 寻址空间不 $\div 8$ （误用带宽的 $\div 8$ ）

- 按字节编址：地址数 = 字节数。20 根地址线 $\rightarrow 2^{20} = 1\text{MB}$ （不是 128KB！）
- $\div 8$ 只出现在 带宽 = 频率 $\times$ 位宽 / 8（bit $\rightarrow$ Byte）；寻址范围是“多少个编号”，编号本身就是字节数，不除 8。

3. 总线定时 同步 vs 异步

- 同步定时：收发靠 公共时钟 对齐；缺点——时钟周期按 最慢器件 设，速度差大的器件间效率低。
- 异步定时：靠 握手信号（请求/应答）逐个确认，快慢自适应，但有额外握手延迟。

4. 总线仲裁的目的（别答成“公平”）

- 根本目的：总线是 共享介质、同一时刻只能一个主设备驱动，仲裁决定这段时间给谁用、按什么顺序。

- 两大类：**集中式**（链式查询 / 计数器定时查询 / 独立请求）vs **分散式**。
- "防止部分设备永远得不到"是 **公平性**，是附带考量不是根本目的。

5. 四种 I/O 控制方式排序 + 数据是否经 CPU

方式	CPU 干的活	数据经 CPU?
程序查询	全程忙等轮询	是
程序中斷	每传一个字 CPU 搬一次	是
DMA	只在预处理+后处理介入、成块搬运	否
通道	执行通道程序独立管理、管得更宽	否

🔗 记忆钩：分界线：查询/中断数据经 CPU，DMA/通道不经过。CPU 负担递减：
查询 > 中断 > DMA > 通道。

6. 中断现场含 PSW + 响应时机

- **现场 = PC（断点）+ PSW（程序状态字）+ 通用寄存器**。只答 PC 漏 PSW 是错的。
- 硬件隐指令保存 **PC + PSW**（断点信息），OS 中断服务程序保存 **通用寄存器**（硬件存断点、软件存现场）。
- **响应时机**：只能在 **当前指令执行完毕**（中断周期）统一查询响应，**不能在指令中途** 立即响应。
- 优先级用于 "**多个中断同时来**" 判先后，跟 "什么时候响应" 是两码事（曾把二者答混）。

7. I/O 接口六功能（漏最核心两条）

- ① **缓冲—速度匹配**（最重要）② **串/并转换—格式匹配** ③ 电平转换 ④ 设备选择(译码)
⑤ 控制与状态 ⑥ 中断/DMA 联络。

🔗 记忆钩：缓冲解决"速度不匹配"、串并解决"格式不匹配"——这两条最常被漏。

8. 寻址方式访存次数 (S1)

寻址	访存次数	理由
寄存器	0	操作数在 CPU 里
直接	1	地址就在指令里，一次取数
寄存器间接	1	寄存器给地址，访存 1 次取数
一次间接	2	先访存取有效地址，再访存取数

⚠ 陷阱：铁律：读寄存器不算访存，只数访问"主存"的次数。

9. 变址 / 基址 / 相对寻址 (S1)

- **变址** $EA = (IX) + A$ ：IX 用户可改、A 固定 → 面向 **数组、循环**。
- **基址** $EA = (BR) + A$ ：BR 由 **OS** 改、A 不变 → 面向 **重定位、多道**。
- **相对** $EA = (PC) + A$ → 面向 **程序浮动、跳转**。

🔗 记忆钩：变址面向数组、基址面向系统、相对面向跳转。

10. 二地址指令结果放哪 + 零地址

- $ADD\ R1,\ R2 \rightarrow$ 结果存 **R1** (覆盖原值)，**R2 不变**；被改的是 R1 和 PC。"结果放 R2、R2 消失"错。
- **零地址** 典型：栈运算指令、NOP/停机。转移指令属 **一地址**，不是零地址。

11. 取指周期数据通路

$(PC) \rightarrow MAR \rightarrow \text{地址总线} \rightarrow \text{主存} \rightarrow MDR \rightarrow IR$ ，再由 IR 的 **操作码段** 送指令译码器；PC 取指末自动 +1。

🔗 记忆钩：PC 出发、MAR 带路、MDR 载货、IR 收货，不是"PC 直送译码器"。

12. 流水线三冒险含控制冒险

数据冒险（nop/旁路/调序）、结构冒险（加硬件）、控制冒险（转移分支 → 分支预测）。常漏控制冒险。

13. 扩展操作码目的

在指令字长固定前提下让操作码长度可变，实现"操作码数量 ↔ 地址码数量"的权衡（地址码少的指令让出更多位给操作码），不是简单"解决操作码太少"。

批次	内容	一刷	二刷（只刷错题）
D1	统考真题 10 道	☒	☐
D2	数据表示 13 道	☒	☐
D3	存储系统 19 道	☒	☐
D4	I/O 20 道	☒	☐
D5	CPU 上 13 道	☒	☐
D6	CPU 下 13 道	☒	☐
D7	总线+指令系统+概述 12 道	☒	☐
D8	近期费曼新增血点 co-04~07（总线/IO/寻址/取指/冒险）	☐	☐

✅ **2026-09-01：D1-D7 一刷全部完成**——93 题一轮过完，薄弱点已全部进手册。⌚ 待办：二刷验收（约 38 道错题）。

打印提示：浏览器打开本文件 → Ctrl+P → 目标选"另存为 PDF" → 边距默认 → 勾选"背景图形"（保留彩色分章）→ 保存。建议 A4 纵向。

计组薄弱点背诵手册 · 2026-09-01 创建 · 随复习持续追加 · 错题本同步：网站 → 错题本 → 408计算机 → 计组